B VE B+ AĞAÇLARININ GÜNÜMÜZDE KULLANIM ALANLARI

BİLAL GÜNEŞ

1. Giriş

B-Tree ve B+ Tree, büyük veri kümelerini disk üzerinde verimli biçimde depolamak ve sorgulamak amacıyla geliştirilmiş dengeli ağaç veri yapılarıdır. 1972 yılında Rudolf Bayer ve Edward McCreight tarafından önerilen bu yapılar, günümüzde veri tabanı sistemleri, dosya sistemleri ve büyük ölçekli veri platformlarının temel bileşenleri arasında yer almaktadır.

2. Veri Tabanlarında Kullanımı

Veri tabanı sistemleri denince aklıma ilk gelen şey, nasıl bu kadar hızlı sonuç döndürebildikleri oluyor. Milyonlarca kayıt arasından birini saniyenin çok küçük bir diliminde bulabilmek, B-Tree ve B+ Tree sayesinde mümkün. Bu yapılar disk erişimini minimumda tutacak şekilde tasarlandığından, büyük veri tabanlarında bile performans kabul edilebilir seviyede kalıyor.

2.1. MySQL

MySQL'in InnoDB motoru B+ Tree kullanıyor. Primary key ile arama yapıldığında veri doğrudan yaprak düğümde bulunuyor, yani ekstra bir adım gerekmiyor. Secondary index'lerde ise yaprak düğümde verinin kendisi değil, primary key değeri tutuluyor. Bu basit ama akıllıca bir tasarım.

2.1. PostgreSQL

PostgreSQL'de de varsayılan indeks türü B-Tree. İlginç olan şu: Hash veya GIN gibi başka seçenekler de sunuluyor ama çoğu geliştirici yine B-Tree'ye dönüyor. Hem eşitlik hem de aralık sorgularında güvenilir sonuç vermesi bence bunun en büyük sebebi.

2.1. MongoDB

MongoDB NoSQL olduğu için farklı bir indeks yapısı kullanıyor diye düşünmüştüm ama hayır, o da B-Tree kullanıyor. Sonunda anladım ki B-Tree'nin verimi veri modeliyle değil, arama mantığıyla ilgili. Belge tabanlı olsan da fark etmiyor.

3. Dosya Sistemlerinde Kullanımı

Dosya sistemleri konusunda da B-Tree'nin bu kadar yaygın olduğunu araştırırken öğrendim. macOS'un APFS'i, Linux'un ext4'ü ve Windows'un NTFS'i hepsi bu yapıyı kullanıyor. Bir klasörde binlerce dosya olduğunda bile arama hızının düşmemesinin sebebi bu. Bence dosya sistemleri için en kritik özellik şu: ağaç her zaman dengeli kalıyor. Ne kadar dosya eklenirse eklensin, silme ya da ekleme sonrası sistem kendini yeniden düzenliyor. Kullanıcı hiçbir zaman 'klasörüm çok doldu, yavaşladı' demek zorunda kalmıyor.

4. Arama Motorları ve Büyük Veri Sistemleri

Büyük veri platformlarına bakıldığında B-Tree'nin doğrudan değil ama temel alınarak kullanıldığı görülüyor. Mesela Apache Cassandra ve HBase, LSM-Tree adı verilen bir yapı kullanıyor; bu da B-Tree'den türetilmiş, yazma performansına odaklanmış bir yaklaşım. Elasticsearch ise metin aramalarında ters indeks kullanıyor ama bunların disk organizasyonu yine ağaç tabanlı. Özetle büyük veri dünyasında B-Tree'nin kendisi değil, mirasını taşıyan yapılar kullanılıyor. Ama temel mantık hâlâ aynı.

5. Modern Sistemlerde Neden Tercih Edilmektedir?

Araştırdıkça birkaç şeyin öne çıktığını gördüm:

- Disk erişimini azaltıyor: Her düğüm bir disk bloğuna denk geliyor, gereksiz okuma olmuyor.
- Her zaman dengeli: Veri ekleyip silerken performans bozulmuyor.
- Aralık sorguları güçlü: B+ Tree'de yaprak düğümler zincirlendiğinden, belirli bir aralıktaki verilere ulaşmak çok hızlı.
- Onlarca yıldır test edilmiş: Güvenilirliği tartışmasız, alternatiflerin henüz geçemediği bir eşik bu.

6. Günlük Hayattan Örnek: Kütüphane Sistemi

B-Tree'yi anlamak için kütüphane örneği bence çok işe yarıyor. Yüz binlerce kitabı olan bir üniversite kütüphanesi düşünelim. Her kitabın konusu, yazarı, ISBN'i var ve bunlar belirli bir düzene göre raflara dizilmiş.

B-Tree burada katalog gibi çalışıyor. Her düğüm birden fazla anahtar tutabiliyor ve sizi doğru alt bölüme yönlendiriyor; tıpkı 'M-P arası kitaplar şu rafta' gibi. Böylece tüm rafları dolaşmak zorunda kalmıyorsunuz.

B+ Tree'de ise yapraklar birbirine bağlı. Bu, '2010-2015 yılları arasında basılan tüm kitaplar' gibi bir sorgu yapıldığında çok işe yarıyor. İlk kitabı bulduktan sonra zinciri takip edip geri kalanlarına sırayla ulaşabiliyorsunuz. Sanki rafların sonunda 'bir sonraki raf burası' yazan bir yön levhası var.

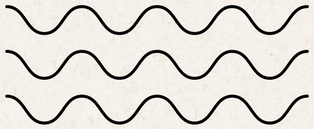
Bir de şunu ekleyeyim: kütüphaneye her gün yeni kitap geliyor, bazıları kayboluyor. B-Tree her değişiklikte kendini yeniden dengelerken katalog hiç karışmıyor. Gerçek sistemlerde de tam olarak bu oluyor.

Bu Veri Yapıları Neden Önemlidir?

Kullandığımız uygulamaların hızlı çalışması, arka planda bu yapıların doğru kullanılmasıyla doğrudan ilişkili. Bankacılık uygulaması hesap geçmişini anlık getiriyorsa, muhtemelen B+ Tree indeksi bunda rol oynuyor. O(log n) karmaşıklığının verdiği garanti olmasa, büyük sistemlerin bu ölçekte çalışması mümkün olmazdı

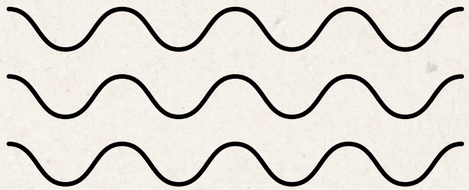
Büyük Veri Çağında Neden İhtiyaç Duyulmaktadır?

1 milyon kayıtta 20 adımda bulunan veri, 1 milyar kayıtta yaklaşık 30 adımda bulunuyor. Bu fark inanılmaz küçük. Düz liste gibi bir yapıda bu sayı milyara çıkardı. Büyük veri dediğimizde sadece veri miktarı değil, hız da önemli; B-Tree ikisini aynı anda çözüyor.



Gelecekte de Kullanılmaya Devam Edecek Mi?

Bence evet, ama dönüşerek. 'Learned index structures' denen, makine öğrenmesiyle B-Tree'yi ikame etmeyi hedefleyen çalışmalar var. Belki ileride bu yaklaşımlar öne geçer. Ama şu an için B-Tree'nin sunduğu öngörülebilir performans ve güvenilirliği hiçbir alternatif tam karşılamıyor. 1970'lerden beri ayakta kalan bir yapı, bu süreyi hak etmeden geçiremez.



Teşekkürler